

ADVANCED EMBEDDED LINUX + MINI iLO SERVICE — COMPLETE COMMAND PURPOSE GUIDE

MAIN GOAL

Build a firmware-style ARM Linux system with:

Component	Purpose
QEMU	ARM hardware emulator
Buildroot	Embedded Linux generator
U-Boot	ARM bootloader
systemd	Linux service manager
Mini iLO Service	Monitoring daemon
journald	Central logging system

Final Result:

Mini iLO-style Embedded Linux Platform
running inside ARM QEMU

COMPLETE SYSTEM FLOW

Ubuntu Host



Buildroot Embedded Linux



ARM Kernel + RootFS



QEMU ARM Hardware



U-Boot Bootloader



systemd Init System



Mini iLO Service



journald Logs

STEP 1 — START EMBEDDED LINUX

```
cd ~/ilo-training/buildroot
```

PURPOSE

Moves terminal into Buildroot workspace.

WHY THIS IS NEEDED

Generated boot images are stored inside:

```
output/images/
```

START ARM LINUX

```
./output/images/start-qemu.sh
```

MAIN PURPOSE

Boots complete ARM Embedded Linux inside QEMU.

WHAT HAPPENS INTERNALLY

QEMU starts ARM virtual hardware



Loads Linux kernel



Loads root filesystem



Starts Linux init system



Displays login prompt

WHY THIS SCRIPT IS IMPORTANT

Buildroot auto-generated correct:

Configuration	Status
Kernel arguments	✓
RootFS mount	✓
Device tree	✓
QEMU machine config	✓
Console binding	✓

STOP OLD QEMU SESSION

kill -9 %1

PURPOSE

Forcefully terminates background QEMU process.

PURPOSE OF EACH PART

Part	Meaning
<code>kill</code>	Sends signal to process
<code>1</code>	
<code>-9</code>	Immediate force termination
<code>%1</code>	Background job ID

WHY THIS IS NEEDED

Old QEMU process may continue using:

- Terminal
 - Serial console
 - VM memory
-

STEP 2 — LOGIN

At login prompt:



buildroot login:

Type:

root

PURPOSE

Logs into Embedded Linux shell.

WHY ROOT USER?

Embedded Linux systems commonly use root-only environments for:

- Firmware debugging
 - Recovery systems
 - BMC firmware
 - Bootloader testing
-

STEP 3 — VIEW RUNNING PROCESSES

ps

PURPOSE

Displays active Linux processes.

WHAT HAPPENS INTERNALLY

Linux reads process data from:

/proc

filesystem.

WHY THIS IS IMPORTANT

Confirms:

Linux userspace started successfully

STEP 4 — CREATE MINI iLO SERVICE SCRIPT

```
cat > /usr/bin/ilo-service << 'EOF'
#!/bin/sh
```

```
while true
do
    echo "Mini iLO Service Running"
    date
    sleep 5
done
EOF
```

MAIN PURPOSE

WHAT THIS SIMULATES

Real firmware services in:

- Hewlett Packard Enterprise iLO
 - Routers
 - Automotive ECUs
 - IoT gateways
-

PURPOSE OF EACH PART

Part	Meaning
<code>cat > file</code>	Create/write file
<code>/usr/bin/ilo-service</code>	Service executable path
<code><< EOF</code>	Multi-line file input
<code>#!/bin/sh</code>	Use shell interpreter
<code>while true</code>	Infinite service loop
<code>echo</code>	Print message
<code>date</code>	Show current time
<code>sleep 5</code>	Wait 5 seconds

INTERNAL EXECUTION FLOW

Linux shell



↓
Executes script
↓
Infinite monitoring loop
↓
Prints logs repeatedly

MAKE SCRIPT EXECUTABLE

`chmod +x /usr/bin/ilo-service`

PURPOSE

Adds execute permission to service script.

WHAT HAPPENS INTERNALLY

Linux modifies:

file permission bits

WHY THIS IS REQUIRED

Without execute permission:

Permission denied

error occurs.

RUN SERVICE

/usr/bin/ilo-service

PURPOSE

Starts mini monitoring daemon.

EXPECTED OUTPUT

Mini iLO Service Running
Mon May ...

every 5 seconds.

STOP SERVICE

Press:

CTRL + C

PURPOSE

Interrupts running foreground process.

WHAT HAPPENS INTERNALLY

Shell sends SIGINT signal



Process terminates

STEP 5 — CLONE U-BOOT

```
cd ~/ilo-training
```

Moves into workspace.

```
git clone https://github.com/u-boot/u-boot.git
```

WHAT IS U-BOOT?

U-Boot is:

Embedded Linux bootloader

Equivalent to BIOS/UEFI on PCs.

WHAT U-BOOT DOES

Responsibility	Purpose
Initialize hardware	CPU/RAM setup
Load Linux kernel	Start OS

Pass boot
arguments

Configure kernel

Load device tree

Describe
hardware

ENTER U-BOOT DIRECTORY

```
cd u-boot
```

STEP 6 — CONFIGURE U-BOOT

```
make CROSS_COMPILE=arm-linux-gnueabi- qemu_arm_defconfig
```

PURPOSE

Loads ARM QEMU default board configuration.

PURPOSE OF EACH PART

Part	Meaning
<code>make</code>	Build system
<code>CROSS_COMPILE=</code>	ARM toolchain prefix
<code>qemu_arm_defconfig</code> <code>ig</code>	ARM QEMU board config

WHAT HAPPENS INTERNALLY

U-Boot build system



Loads ARM config



Creates .config

STEP 7 — BUILD U-BOOT

```
make -j$(nproc) CROSS_COMPILE=arm-linux-gnueabihf-
```

PURPOSE

Cross compiles ARM bootloader.

WHAT HAPPENS INTERNALLY

U-Boot source



ARM compiler



ARM bootloader binary

INSTALL REQUIRED DEPENDENCY

```
sudo apt install libgnutls28-dev
```

PURPOSE

Installs TLS/SSL development libraries.

WHY THIS IS NEEDED

Some U-Boot features require secure networking libraries.

VERIFY GENERATED FILES

Is u-boot*

PURPOSE

Lists generated U-Boot binaries.

IMPORTANT OUTPUTS

File	Purpose
<code>u-boot</code>	ELF executable
<code>u-boot.bin</code>	Raw bootloader image

STEP 8 — RUN U-BOOT INSIDE QEMU

`qemu-system-arm -M versatilepb -m 256M -kernel u-boot -serial mon:stdio`

PURPOSE

Boots U-Boot using ARM QEMU emulator.

PURPOSE OF EACH ARGUMENT

Argument	Meaning
<code>qemu-system-arm</code>	ARM machine emulator
<code>-M versatilepb</code>	ARM board type
<code>-m 256M</code>	RAM size
<code>-kernel u-boot</code>	Load U-Boot
<code>-serial mon:stdio</code>	UART console to terminal

VERY IMPORTANT CONCEPT

UART = Serial communication console

Used heavily in:

- Embedded Linux
 - iLO/BMC
 - Routers
 - Automotive firmware
-

STEP 9 — BOOT FULL SYSTEM USING U-BOOT

```
qemu-system-arm \  
-M versatilepb \  
-m 256M \  
-bios u-boot/u-boot.bin \  
-kernel buildroot/output/images/zImage \  
-dtb buildroot/output/images/versatile-pb.dtb \  
-drive file=buildroot/output/images/rootfs.ext2,if=scsi,format=raw \  
-append "root=/dev/sda console=ttyAMA0 rw" \  
-nographic
```

MAIN PURPOSE

Boot complete ARM Linux system using:

- U-Boot
 - Linux kernel
 - Root filesystem
 - Device tree
-

COMPLETE BOOT FLOW

QEMU ARM Hardware



U-Boot Bootloader



Linux Kernel



Device Tree



RootFS Mount



systemd/BusyBox



↓
Linux Login

PURPOSE OF IMPORTANT ARGUMENTS

Argument	Purpose
<code>-bios</code>	Load U-Boot
<code>-kernel</code>	Linux kernel
<code>-dtb</code>	Hardware config
<code>-drive</code>	Root filesystem
<code>console=ttyAM A0</code>	UART console
<code>-nographic</code>	Terminal mode

ERROR — UNABLE TO MOUNT ROOT FS

WHY THIS HAPPENS

Linux kernel cannot locate filesystem.

Possible causes:

Cause	Meaning
Wrong root device	<code>/dev/sda</code> mismatch
Missing filesystem driver	ext2/ext4 absent

DEBUG COMMANDS

```
uname -a
cat /proc/cpuinfo
mount
ps
dmesg | tail
```

PURPOSE OF EACH COMMAND

Command	Purpose
<code>uname -a</code>	Kernel details
<code>cpuinfo</code>	ARM CPU details
<code>mount</code>	Mounted filesystems
<code>ps</code>	Running processes
<code>dmesg</code>	Kernel boot logs

STEP 10 — ENHANCED MINI iLO SERVICE

```
cat > /usr/bin/ilo-service << 'EOF'
#!/bin/sh
```

```
while true
do
    echo "mini iLO monitoring service"
    date
    uptime
    sleep 5
```

done
EOF

IMPROVEMENT

Adds:

uptime

to monitor:

- System load
 - System running time
-

STEP 11 — ENABLE SYSTEMD

cd ~/ilo-training/buildroot

OPEN CONFIG MENU

make menuconfig

WHAT IS menuconfig?

Interactive Linux configuration interface.

PURPOSE

Modify Buildroot settings.

INSIDE MENUCONFIG

Navigate to:

System configuration

Find:

Init system

WHY THIS IS IMPORTANT

Linux requires init system.

Options include:

Init System	Purpose
BusyBox init	Lightweight init
systemd	Advanced service manager

CLEAN OLD BUILD

make clean

PURPOSE

Removes previous build artifacts.

WHY REQUIRED?

Changing init system requires complete rebuild.

REBUILD SYSTEM

```
make -j$(nproc)
```

PURPOSE

Rebuilds Linux system with systemd support.

START NEW SYSTEM

```
./output/images/start-qemu.sh
```

VERIFY SYSTEMD

```
ps
```

EXPECTED RESULT

systemd

running as PID 1.

VIEW LOGS

journalctl

PURPOSE

Displays system logs from journald.

EXIT JOURNAL

Press:

q

FIND QEMU PROCESS

ps -ef | grep qemu

PURPOSE

Find running QEMU virtual machine processes.

KILL PROCESS

```
kill -9 1178004
```

PURPOSE

Forcefully terminate stuck QEMU process.

STEP 12 — CREATE SYSTEMD SERVICE

START SYSTEM AGAIN

```
./output/images/start-qemu.sh
```

CREATE SERVICE SCRIPT

```
cat > /usr/bin/ilo-service << 'EOF'
```

```
#!/bin/sh
```

```
while true
```

```
do
```

```
    echo "mini iLO service running"
```

```
    date
```

```
    sleep 5
```

```
done
```

MAKE EXECUTABLE

```
chmod +x /usr/bin/ilo-service
```

CREATE SYSTEMD UNIT FILE

```
cat > /etc/systemd/system/ilo.service << 'EOF'
```

```
[Unit]
```

```
Description=Mini iLO Service
```

```
After=network.target
```

```
[Service]
```

```
ExecStart=/usr/bin/ilo-service
```

```
Restart=always
```

```
[Install]
```

```
WantedBy=multi-user.target
```

```
EOF
```

WHAT IS SYSTEMD UNIT FILE?

Configuration describing Linux service behavior.

PURPOSE OF EACH SECTION

Section	Purpose
---------	---------

[Unit] Metadata/dependencies

[Service] Execution details
]

[Install] Boot startup rules
]

IMPORTANT PARAMETERS

Parameter	Purpose
ExecStart	Service executable
Restart=always	Auto restart service
WantedBy=	Auto-start target

RELOAD SYSTEMD

systemctl daemon-reload

PURPOSE

Reload systemd configuration files.

ENABLE SERVICE

systemctl enable ilo.service

PURPOSE

Start service automatically during boot.

START SERVICE

```
systemctl start ilo.service
```

PURPOSE

Launch service immediately.

CHECK SERVICE STATUS

```
systemctl status ilo.service
```

PURPOSE

Displays:

- Running state
 - PID
 - Service logs
 - Restart status
-

VIEW SERVICE LOGS

`journalctl -u ilo.service`

PURPOSE

Displays logs only for ilo.service.

INTERNAL SYSTEMD FLOW

systemd
↓
Reads ilo.service
↓
Executes ilo-service
↓
Monitors daemon
↓
Captures logs
↓
Stores logs in journald
↓
journalctl displays logs

FINAL INDUSTRY UNDERSTANDING

This architecture is similar to:

Industry System

Equivalent

Hewlett Packard Enterprise iLO Monitoring daemon

Router firmware	Linux service
Automotive ECU	Embedded daemon
IoT firmware	systemd agent
BMC firmware	Logging + monitoring

FINAL COMPLETE UNDERSTANDING FLOW

